

Mapping Data Models to VOTable

Omar Laurino,¹ Gerard Lemson,² Mark Cresitello-Dittmar,¹ Tom Donaldson,³
and Laurent Michel⁴

¹*Smithsonian Astrophysical Observatory, Cambridge, MA, USA;*
olaurino@cfa.harvard.edu

²*Johns Hopkins University, Baltimore, MD, USA*

³*Space Telescope Science Institute, Baltimore, MD, USA*

⁴*Observatoire de Strasbourg, Strasbourg, France*

Abstract. Data providers and curators provide a great deal of metadata with their data files. This metadata is invaluable for users and for Virtual Observatory software developers. In order to be interoperable, the metadata must refer to common data models. A new specification is being developed by the IVOA Data Modeling Working Group to define a scheme for annotating VOTable instances in a standard, consistent, interoperable fashion, so that each piece of metadata can unambiguously refer to the correct data model element it expresses. The mapping can be extended to formats other than VOTable.

1. Use Cases

The use cases for a serialization format may be rather abstract and technical, because the format is generally agnostic of the data that it will encode. However, a number of domain specific constraints can be taken into account to drive the format's design. Similarly, one can spell out exemplary domain use cases that make it explicit what kind of usage the serialization format addresses.

In the most abstract sense the main use case is to provide a complete, unambiguous, interoperable means to encode and decode instances of structured data models in VOTable and other existing formats. This is something that the current formats do not allow, or at least not in an interoperable and unambiguous fashion, or they are not expressive enough to represent complex structured instances (Thomas et al. 2015; Graham et al. 2013). Moreover, the format must be extensible, in the sense that users must be able to express instances of new models with a single standard, rather than having to come up with a standard representation for each model.

Note that the above are rather strong constraints, because they entail that the format requires a *mapping* between two existing representations. One representation defines the model and its instances, the other is the existing data format, e.g. VOTable.

The constraints above lead to an unusual data modeling workflow. Typically an abstract representation of a model is then translated to a physical, specific representation through object oriented software, data base schemata, or XML schemata. In our case we are looking for a way to *annotate* data and metadata in existing formats with their

own pre-existing schemata, so that such data and metadata can be interpreted in terms of complex concepts in a generic, interoperable, shared conceptual data model.

A typical usage scenario may be a VOTable client that is sensitive to certain models only, say coordinates, measurements, and World Coordinate System transforms. Such a client may be written to understand annotations for coordinates and frames, manipulate such instances, and write them back to disk. The actual representation is usually tabular, with e.g. coordinates expressed as cells in a table whose columns represent several properties of a source, including its position.

More complex models for astronomical data products may enable smart plotting and fitting applications. As high-level models for Data Cubes, Time Series, or Spectra use the same building blocks for measurements and coordinates an application may discover these pieces of information and structure a plot, or perform a fit, or create an animation, with minimal user input, together with features for data validation, units conversion, intelligent projections or domain specific calculations.

Having a generic standard for annotating VOTable and other tabular formats can be useful even when clients are not necessarily aware of all or even any of the models mapped in a data file. For instance, data discovery portals may provide a friendly interface that allows users to select or query by physical quantities using standardized, structured representations. It may do so dynamically for all the pieces of metadata present in the dataset, rather than limiting functionality to a set of hard-coded metadata properties.

Moreover, the existence of an explicit data model representation language and of a precise mapping specification enables the creation of universal validators, just as it happens for XML and XSD: the validator may parse the data model descriptions declared by the VOTable and check that the file represents valid instances of one or more data models.

While most use cases enabled by this specification may help clients implementing useful features for users, the definition of a standard for serializing data model instances can lead to the development of tools for publishers to build templates of valid responses. The mapping specification thus provides data publishers with a framework for expressively and accurately representing their data holdings in an interoperable way, thus leveraging all the work that goes into maintaining and curating their data and metadata.

2. Mapping Data Models to VOTable

Designing an interoperable scheme for representing structured objects in multiple contexts interoperably leads to the challenges usually associated with Object-Relational Mapping.

The VO Data Modeling Language (Lemson et al. 2018) defines a very simple XML schema for representing data models as XML documents¹. VODML maps directly to a very small subset of UML, so that each data model can be represented by a simple UML class diagram. Instances of data model types could thus be represented by UML object diagrams, although that is not usually useful and certainly not part of the standard. In practice such instances are usually represented in relational databases,

¹Note how this is different from the usual workflow where data models *are* indeed represented by XML schemata, rather than document

object-oriented software, FITS, VOTable, and other tabular or non-tabular representations.

The inherent complexity of Object-Relational mapping is mitigated by the fact that in most cases astronomical data sets are presented to the user in a rather denormalized, flattened fashion as a single table. This allows for a simple annotation for simple cases, which are the vast majority.

Complex representations with multiple tables must still be supported for more complex cases, especially for those where instances in a table may refer to instances in other tables. While this is not a common scenario, there are indeed complex data products where a single table can, for instance, contain multiple time series.

VODML Data Models define *Types* and *Roles*, e.g. `EquatorialCoordinate` is a *Type* with *Roles* `ra`, `dec`, and `frame`. Models define globally unique, portable identifiers for types and roles, e.g.² `coords:domain.space.EquatorialCoord` and `coords:domain.space.EquatorialCoord.frame`.

These portable identifiers can be used to map data and metadata in VOTable to the concept they represent, in an unambiguous fashion. The full specification of how to perform such mapping is the subject of the Mapping Models to VOTable standard (Lemson et al. 2017).

The mapping specification defines an extension to the VOTable v1.3 schema that provides hooks for data producers to annotate their tabular data files.³

The annotation for the full file is all contained in a single *VODML* block, so it's easy for non-VODML or legacy clients to simply ignore the new annotations. Additionally, data producers can add annotations without changing their existing files or services. Indeed, one could even add the *VODML* element with its annotations programmatically in order to wrap existing data files, given that a mapping is performed once for each class of data files. Enabling such mapping of existing services or files is part of the reference implementations described in the next section.

At the time of this writing VODML is an actual IVOA standard, while the Mapping standard is currently in its Working Draft stage, as implementations inform changes and design decisions to the first drafts of the document.

3. Implementations

A web page (<https://olaurino.gitlab.io/ivoa-dm-examples>) collects information and demonstrations of many such implementations⁴, including data models, software, and notebooks demonstrating the interoperability of independent reference implementations on the server and client side. The serializations include coordinates, measurements, and time series as sparse data cubes.

²These examples are based on current models but they are not necessarily actual identifiers from actual models. They are just realistic examples.

³An equivalent mapping is possible with the basic VOTable 1.3 schema only. However, after an initial round of implementations the working group decided to review the VOTable schema to make the annotations simpler and more flexible.

⁴Given the ongoing nature of these implementations, the best pointers we can provide at this point are URLs

A service providing VODML-annotated files was developed at the Space Telescope Science Institute as part of a development version of the MAST portal, while a Python client (<https://github.com/olaurino/rama>) implementing the Mapping specification was developed at the Chandra X-Ray Center.

The Python client (Rama) offers a framework for translating VO standard representations as Python objects, with a specific focus on leveraging Astropy (Greenfield et al. 2013) classes and features.

Creating valid annotations can be an error prone process. However, the existence of consistent standards for both model descriptions and annotations makes it possible to write generic tools that abstract users from standard documents, so to allow them to create annotations using their domain knowledge and the knowledge of their data sets. Two tools have been implemented with very different philosophies, as described below.

Jovial (<https://github.com/olaurino/jovial>) implements both VODML and the Mapping standard to provide data modelers and providers with Domain Specific Languages (DSLs). The first implementation was written in Groovy, while a new implementation in Kotlin, with better support for Integrated Development Environments, is underway. Domain Specific Languages can be useful in reducing the complexity of tasks by employing a more human readable and writable syntax for writing programs in a more natural, declarative way.

While DSLs are rather useful because a textual format is easily versionable, shareable, and allows for rapid prototyping, Graphical User Interfaces (GUIs) offer the shortest learning curve possible. For this reason a *drag and drop* GUI was developed for allowing users to easily map standard data models to tables and even services. The application can help users and data providers to create template or even full annotations for their datasets and services while requiring minimal exposure to the standard document themselves.

Acknowledgments. Omar Laurino and Mark Cresitello-Dittmar acknowledge support by NASA under contract NAS 8-03060 to the Smithsonian Astrophysical Observatory for operation of the Chandra X-ray Center. Gerard Lemson is funded by the U.S. National Science Foundation through its Data Infrastructure Building Blocks (DIBBs) program, award ACI-1261715. Tom Donaldson acknowledges support from the NASA Astronomical Virtual Observatories (NAVO), provided by NASA through the Astrophysics Data Curation and Archival Research (ADCAR) program. Laurent Michel is funded by OV France and by the the Survey Science Consortium of XMM Newton.

References

- Graham, M., et al. 2013, UTypes: current usages and practices in the IVOA, Tech. rep.
- Greenfield, P., et al. 2013, Astropy: Community Python library for astronomy, Astrophysics Source Code Library. 1304.002
- Lemson, G., et al. 2017, Mapping Data Models to VOTable, Tech. rep.
- 2018, VO-DML: a consistent modeling language for IVOA data models, Tech. rep.
- Thomas, B., et al. 2015, Astronomy and Computing, 12, 133